

PROJECT REPORT

Project no: 15

Project title:

Design and Development of a Water Level Detection and Flood Alert System using Ultrasonic Sensor

Submitted by:

Arundepak S – 24MER004

Deepak p – 24MER006

Dharanidharan R S – 24MER007

Dinesh K – 24MER009

Vishnuvarthan P – 24MER063

Design and Development of a Water Level Detection and Flood Alert System using Ultrasonic Sensor

1. Project Overview

This project presents the design and development of a Water Level Detection and Flood Alert System built around an ESP32 microcontroller. The system continuously measures the water level in a river, tank, or dam using an HC-SR04 ultrasonic distance sensor mounted above the water surface, converts the measured distance into an equivalent water level, and classifies the reading into SAFE, WARNING, or DANGER zones based on two configurable thresholds. The current level and status are shown locally on a 16x2 I2C LCD, visual indication is given through green and red LED indicators, and an audible buzzer sounds during warning and danger conditions.

In addition to local feedback, a SIM800L GSM module sends an SMS alert to a pre-configured emergency contact number the moment the water level crosses the danger threshold, and all readings are simultaneously pushed over Wi-Fi to the Adafruit IO cloud platform for remote, real-time monitoring. This gives site personnel, facility managers, and downstream residents an early, reliable warning of rising water levels well before an actual overflow or flood occurs, without requiring anyone to be physically present at the river, tank, or dam.

2. Problem Statement

Water levels in rivers, dams, and storage tanks are conventionally monitored using manual inspection or simple float-based gauges. This approach has several limitations:

- Water level is not continuously logged, making it difficult to detect a rapidly rising level in time to respond.
- A sudden rise during heavy rainfall or upstream release can go unnoticed until overflow or flooding has already begun, especially at night or at remote, unattended sites.
- Manual monitoring offers no guaranteed, timely warning to facility staff or downstream residents, increasing the risk of property damage, crop loss, or danger to life.
- There is no automatic historical record of water level trends that could support flood preparedness or maintenance planning.
- Commercial flood-warning systems are often expensive, putting them out of reach for small dams, local irrigation tanks, or community-level deployments.

There is therefore a need for an automated, low-cost, wireless system that can continuously measure water level, raise an immediate local and remote alarm as the level rises, notify a responsible person directly by SMS, and make all readings remotely accessible for monitoring and analysis.

3. Proposed Solution

The proposed system uses an ESP32 development board as the central controller, interfaced with an HC-SR04 ultrasonic sensor, a buzzer, two LED indicators, a 16x2 I2C LCD, and a SIM800L GSM module.

Key features of the solution:

- Continuous, non-contact measurement of water level using an HC-SR04 ultrasonic sensor mounted above the maximum expected water level.
- Three-zone classification of the reading — SAFE, WARNING, and DANGER — based on two configurable distance thresholds.
- On-site 16x2 I2C LCD display showing the current water level and status for immediate readability by site personnel.
- Dual LED indicators (green for SAFE, red for DANGER) and a buzzer that beeps intermittently during WARNING and sounds continuously during DANGER.
- Automatic SMS alert sent once via the SIM800L GSM module as soon as the DANGER threshold is crossed, with a latch flag to avoid repeatedly resending the message while the level remains critical.
- Wireless upload of water level and flood status to the Adafruit IO cloud every 5 seconds, respecting the platform's free-tier rate limits.
- Remote dashboard on Adafruit IO with a live gauge, a status indicator, and a trend chart, allowing the water level to be monitored from anywhere.

4. System Architecture

The system follows a three-layer IoT architecture:

- Perception Layer: The HC-SR04 ultrasonic sensor measures the distance from the sensor to the water surface.
- Processing / Edge Layer: The ESP32 converts the measured distance into a water level, evaluates the SAFE / WARNING / DANGER threshold logic, and drives the LCD, LEDs, and buzzer accordingly.
- Network & Cloud / Alert Layer: The ESP32 sends water-level and flood-status data over Wi-Fi to the Adafruit IO cloud platform, and independently triggers the SIM800L GSM module to send an SMS alert over the cellular network when the danger threshold is crossed.

Data flow: HC-SR04 Sensor → ESP32 (distance-to-level conversion, threshold logic) → LCD / LEDs / Buzzer (local feedback) and SIM800L GSM (SMS alert) → Wi-Fi → Adafruit IO Cloud (remote monitoring).

5. Circuit Table

The table below lists the physical wiring between each module and the ESP32 development board, as implemented in the prototype.

Component	Pin	ESP32 Pin	Function
HC-SR04 Ultrasonic Sensor	TRIG	GPIO 5	Trigger pulse output
HC-SR04 Ultrasonic Sensor	ECHO	GPIO 18	Echo pulse input (via voltage divider)
Buzzer	Signal (+)	GPIO 25	Audible alarm
Green LED	Anode	GPIO 26	SAFE indicator
Red LED	Anode	GPIO 27	DANGER indicator
I2C LCD (16x2)	SDA	GPIO 21	Display data
I2C LCD (16x2)	SCL	GPIO 22	Display clock
SIM800L GSM Module	TXD	GPIO 16 (RX2)	GSM → ESP32 serial data
SIM800L GSM Module	RXD	GPIO 17 (TX2)	ESP32 → GSM serial data (via voltage divider)
All Modules	VCC / GND	3V3–5V / G	Power

6. Circuit Design

The HC-SR04 sensor, buzzer, LED indicators, I2C LCD, and SIM800L GSM module are wired to the ESP32 as per the circuit table above, with common power and ground rails distributed via the breadboard. The HC-SR04's ECHO line and the SIM800L's RXD line are stepped down through a voltage divider before reaching the ESP32's 3.3 V-tolerant GPIO pins, since both modules can output signals above the ESP32's safe input level. The LCD communicates over I2C using GPIO21 (SDA) and GPIO22 (SCL), while the GSM module communicates with the ESP32 over hardware UART2 on GPIO16 (RX2) and GPIO17 (TX2).

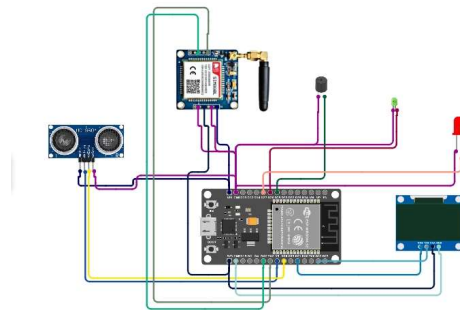


Figure 1: Circuit wiring diagram — ESP32 with HC-SR04 ultrasonic sensor, SIM800L GSM module, buzzer, LED indicators, and I2C LCD

7. Algorithm

Step-by-step operating logic implemented in firmware:

1. Initialize Serial (115200 baud) for debugging and hardware Serial2 (9600 baud) on GPIO16/17 for the SIM800L GSM module; configure the sensor, buzzer, and LED pins; initialize the I2C LCD and display a startup message.
2. Connect to Adafruit IO, which internally manages the Wi-Fi connection, and block until the connection status reaches AIO_CONNECTED, updating the LCD to show “Cloud Connected”.
3. Send an AT command to the SIM800L GSM module and read back its response to confirm the module is present and responding.
4. In the main loop, call io.run() on every iteration to keep the Adafruit IO connection alive.
5. Trigger the HC-SR04 with a 10 μ s pulse and measure the echo pulse width with a 30 ms timeout; compute distance = duration $\times$ 0.0343 / 2, and flag a sensor error if no echo is received.
6. Compute the water level as (sensor mounting height – measured distance), clamped to a minimum of zero.
7. Compare the computed water level against the SAFE threshold (60 cm) and the DANGER threshold (85 cm) to classify the reading as SAFE, WARNING, or DANGER.
8. SAFE: turn the green LED on, the red LED and buzzer off, update the LCD, and reset the SMS-alert latch.
9. WARNING: turn both LEDs off, pulse the buzzer on and off at 300 ms intervals, update the LCD, and keep the SMS-alert latch reset.
10. DANGER: turn the red LED on, hold the buzzer on continuously, update the LCD to show “DANGER!”, and — only if the SMS-alert latch is not already set — send an SMS via the SIM800L and set the latch so the message is not resent on every loop while the level remains critical.
11. Every CLOUD_UPDATE_INTERVAL (5 seconds), publish the water level and status string to the Adafruit IO water-level and flood-status feeds, to stay within the platform's free-tier rate limits.
12. Repeat the loop indefinitely with a 1-second delay between readings.

8. Coding

The complete Arduino/ESP32 firmware (C++) implementing the algorithm above is listed below.

```
/* =====  
WATER LEVEL DETECTION AND FLOOD ALERT SYSTEM  
Board: ESP32 DevKit  
Modules: HC-SR04, Buzzer, 2x LED, 16x2 I2C LCD, SIM800L GSM  
Cloud: Adafruit IO (WiFi)  
===== */  
  
#include <Wire.h>  
#include <LiquidCrystal_I2C.h>  
#include <HardwareSerial.h>  
#include <WiFi.h>  
#include "AdafruitIO_WiFi.h"  
  
// ----- WIFI CREDENTIALS -----  
#define WIFI_SSID    "test"
```

```

#define WIFI_PASSWORD "test1234"

// ----- ADAFRUIT IO CREDENTIALS -----
#define AIO_USERNAME "dharanidharan_97969"
#define AIO_KEY      "aio_tbXF62lqhm7SXUJV8ryomFcvgviv"

AdafruitIO_WiFi io(AIO_USERNAME, AIO_KEY, WIFI_SSID, WIFI_PASSWORD);

// Feeds on Adafruit IO
AdafruitIO_Feed *waterLevelFeed = io.feed("water-level");
AdafruitIO_Feed *floodStatusFeed = io.feed("flood-status");

// ----- LCD CONFIG -----
#define LCD_ADDRESS 0x27 // change to 0x3F if 0x27 doesn't work
#define LCD_COLS 16
#define LCD_ROWS 2
LiquidCrystal_I2C lcd(LCD_ADDRESS, LCD_COLS, LCD_ROWS);

// ----- PIN CONFIG (matches ESP32 board silkscreen) -----
#define TRIG_PIN 5 // D5 -> HC-SR04 TRIG
#define ECHO_PIN 18 // D18 -> HC-SR04 ECHO (via voltage divider)
#define BUZZER_PIN 25 // D25 -> Buzzer +
#define GREEN_LED 26 // D26 -> Green LED
#define RED_LED 27 // D27 -> Red LED
#define SDA_PIN 21 // D21 -> LCD SDA
#define SCL_PIN 22 // D22 -> LCD SCL
#define GSM_RX 16 // D16 (RX2) <- SIM800L TXD
#define GSM_TX 17 // D17 (TX2) -> SIM800L RXD (via voltage divider)

HardwareSerial gsmSerial(2); // ESP32 hardware UART2 for GSM

// ----- SYSTEM PARAMETERS -----
const float SENSOR_HEIGHT_CM = 100.0;
const float SAFE_THRESHOLD_CM = 60.0;
const float DANGER_THRESHOLD_CM = 85.0;
const unsigned long LOOP_DELAY_MS = 1000;
const unsigned long CLOUD_UPDATE_INTERVAL = 5000; // send to cloud every 5s

const char* PHONE_NUMBER = "+917806810340";

// ----- STATE VARIABLES -----
bool alertSent = false;
unsigned long lastCloudUpdate = 0;

// =====
// FUNCTION: Measure distance using HC-SR04
// =====
float getDistanceCM() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);

```

```

delayMicroseconds(10);
digitalWrite(TRIG_PIN, LOW);

long duration = pulseIn(ECHO_PIN, HIGH, 30000);
if (duration == 0) return -1;

return duration * 0.0343 / 2.0;
}

void sendSMS(String message) {
gsmSerial.println("AT+CMGF=1");
delay(500);
gsmSerial.print("AT+CMGS=\"");
gsmSerial.print(PHONE_NUMBER);
gsmSerial.println("\");
delay(500);
gsmSerial.print(message);
delay(500);
gsmSerial.write(26);
delay(3000);
}

void updateLCD(float level, String status) {
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Lvl:");
lcd.print(level, 1);
lcd.print("cm");

lcd.setCursor(0, 1);
lcd.print("Status: ");
lcd.print(status);
}

// =====
// FUNCTION: Push data to Adafruit IO
// =====
void updateCloud(float level, String status) {
waterLevelFeed->save(level);
floodStatusFeed->save(status);
Serial.println("Data sent to Adafruit IO");
}

// =====
// SETUP
// =====
void setup() {
Serial.begin(115200);
gsmSerial.begin(9600, SERIAL_8N1, GSM_RX, GSM_TX);

pinMode(TRIG_PIN, OUTPUT);

```

```

pinMode(ECHO_PIN, INPUT);
pinMode(BUZZER_PIN, OUTPUT);
pinMode(GREEN_LED, OUTPUT);
pinMode(RED_LED, OUTPUT);

digitalWrite(BUZZER_PIN, LOW);
digitalWrite(GREEN_LED, LOW);
digitalWrite(RED_LED, LOW);

// I2C for LCD
Wire.begin(SDA_PIN, SCL_PIN);
lcd.init();
lcd.backlight();
lcd.setCursor(0, 0);
lcd.print("Flood Alert Sys");
lcd.setCursor(0, 1);
lcd.print("Connecting WiFi");

// Connect to Adafruit IO (handles WiFi connection internally)
Serial.print("Connecting to Adafruit IO");
io.connect();

while (io.status() < AIO_CONNECTED) {
  Serial.print(".");
  delay(500);
}
Serial.println();
Serial.println(io.statusText());

lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Cloud Connected");
delay(1500);

// Check GSM module response
Serial.println("Checking GSM module...");
gsmSerial.println("AT");
delay(1000);
while (gsmSerial.available()) {
  Serial.write(gsmSerial.read());
}

delay(1000);
Serial.println("System Initialized");
}

// =====
// MAIN LOOP
// =====
void loop() {
  io.run(); // required every loop for Adafruit IO connection

```

```

float distance = getDistanceCM();

if (distance < 0) {
  Serial.println("Sensor Error: No Echo");
  updateLCD(0, "ERROR");
  delay(LOOP_DELAY_MS);
  return;
}

float waterLevel = SENSOR_HEIGHT_CM - distance;
if (waterLevel < 0) waterLevel = 0;

Serial.print("Distance: "); Serial.print(distance);
Serial.print(" cm | Water Level: "); Serial.print(waterLevel);
Serial.println(" cm");

String currentStatus;

if (waterLevel < SAFE_THRESHOLD_CM) {
  // ----- SAFE -----
  digitalWrite(GREEN_LED, HIGH);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER_PIN, LOW);
  currentStatus = "SAFE";
  updateLCD(waterLevel, currentStatus);
  alertSent = false;

} else if (waterLevel < DANGER_THRESHOLD_CM) {
  // ----- WARNING -----
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);
  currentStatus = "WARNING";
  updateLCD(waterLevel, currentStatus);

  digitalWrite(BUZZER_PIN, HIGH);
  delay(300);
  digitalWrite(BUZZER_PIN, LOW);
  delay(300);
  alertSent = false;

} else {
  // ----- DANGER -----
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, HIGH);
  digitalWrite(BUZZER_PIN, HIGH);
  currentStatus = "DANGER";
  updateLCD(waterLevel, "DANGER!");

  if (!alertSent) {

```

```

String msg = "FLOOD ALERT! Water level critical: " + String(waterLevel) + " cm. Immediate action
required.";
sendSMS(msg);
alertSent = true;
Serial.println("SMS Alert Sent");
}
}

// Push to Adafruit IO every CLOUD_UPDATE_INTERVAL (avoid rate limits)
if (millis() - lastCloudUpdate > CLOUD_UPDATE_INTERVAL) {
  updateCloud(waterLevel, currentStatus);
  lastCloudUpdate = millis();
}

delay(LOOP_DELAY_MS);
}

```

9. System Working

On power-up, the ESP32 initializes all peripherals, connects to Adafruit IO over Wi-Fi, and confirms the GSM module is responding, showing each step on the LCD and Serial Monitor. During normal operation, the HC-SR04 is triggered once per second and the returned echo time is converted into a water level reading. As long as the level stays below the SAFE threshold, the green LED stays lit and the buzzer remains silent; once the level enters the WARNING band, both LEDs turn off and the buzzer beeps intermittently as an early caution.

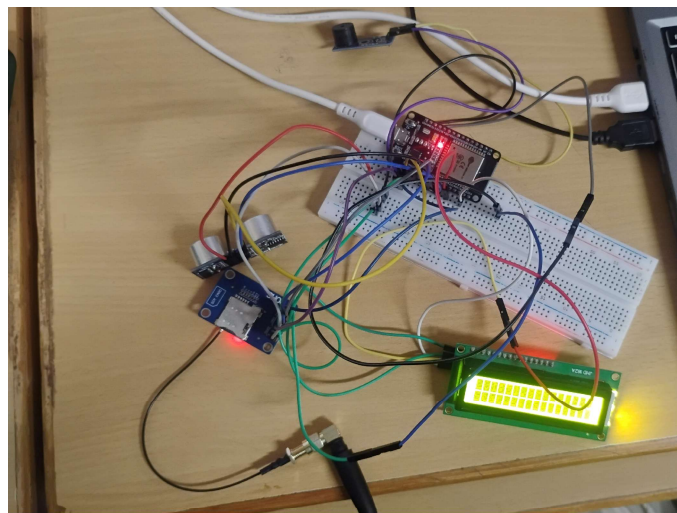


Figure 2: Assembled hardware prototype on breadboard — ESP32, HC-SR04, SIM800L GSM module, and I2C LCD

If the level continues to rise past the DANGER threshold, the red LED turns on, the buzzer sounds continuously, and the system immediately sends a one-time SMS alert to the configured emergency contact number over the SIM800L GSM module — independent of the Wi-Fi/cloud connection, so the alert still goes out even if the local network is unavailable. In parallel, the water level and status are pushed to the

Adafruit IO cloud every 5 seconds, updating a live gauge, a flood-status indicator, and a trend chart, so the system can be monitored remotely from any browser. This closes the loop between physical sensing, local alerting, SMS notification, and cloud-based monitoring.

10. Bill of Materials

Component	Quantity	Purpose
ESP32 Development Board	1	Main microcontroller — sensor processing, Wi-Fi/cloud, GSM control
HC-SR04 Ultrasonic Sensor	1	Non-contact water level (distance) measurement
SIM800L GSM Module	1	Sends SMS flood alert over the cellular network
16x2 I2C LCD Display	1	On-site display of water level and status
Buzzer	1	Audible local alarm
LED Indicators (Green, Red)	2	Visual SAFE / DANGER status indication
Breadboard	1	Prototyping platform for circuit assembly
Jumper Wires	1 Set	Interconnections between modules and ESP32

11. Prototype Implementation

The firmware was developed and uploaded to the ESP32 using Arduino IDE 2.3.10 (board: ESP32 Dev Module) with the LiquidCrystal_I2C and AdafruitIO_WiFi libraries. After flashing, the Serial Monitor was used to verify the calibrated distance and water-level readings, the sensor-error handling, the SMS-alert trigger, and the Adafruit IO upload behaviour, confirming reliable end-to-end operation.

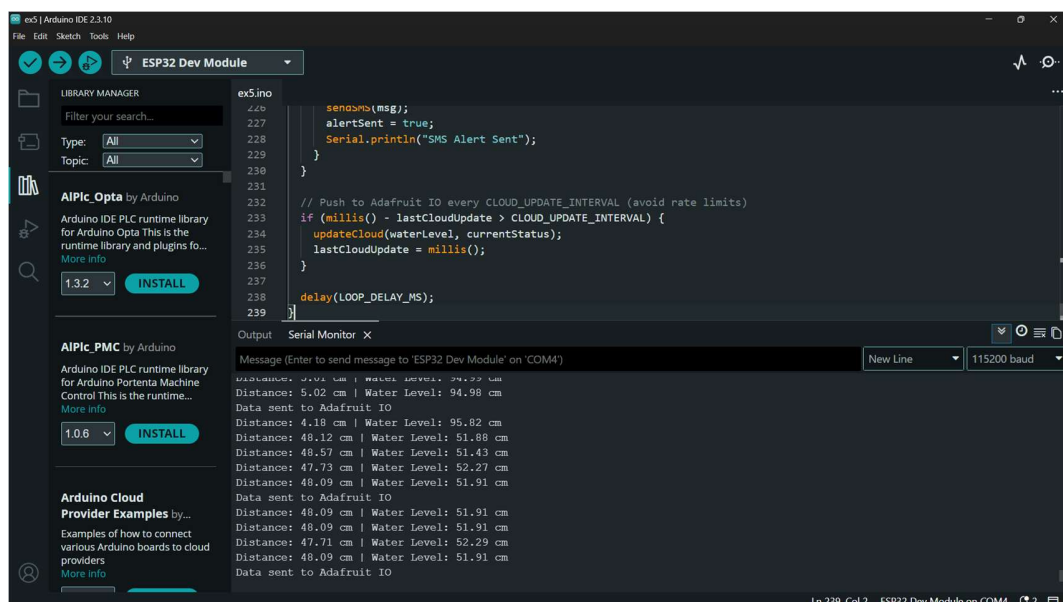


Figure 3: Arduino IDE Serial Monitor — live distance and water-level readings with periodic “Data sent to Adafruit IO” confirmations

12. Results & Performance Analysis

The system was tested continuously over multiple update cycles, with data successfully logged to the Adafruit IO dashboard “GSM Flood Alert System”. The results confirm correct end-to-end functioning:

- **Water Level:** readings tracked consistently with the simulated test distance, settling around 51.4–52.3 cm during SAFE-condition testing and 94.9–96.0 cm during DANGER-condition testing, correctly reflecting the applied sensor distance in both cases.
- **SMS Alert:** the Serial Monitor log recorded a single “SMS Alert Sent” message at the moment the water level first crossed 85 cm (95.54 cm), with no repeat SMS logged on subsequent DANGER readings — confirming the alert-latch logic correctly prevents duplicate messages.
- **Cloud Dashboard:** the Adafruit IO water-level gauge and flood-status widget matched the bench test conditions, reading approximately 52.31 with a green gauge and a “SAFE” status during safe testing, and approximately 97 with a “DANGER” status during danger testing.
- **Upload Reliability:** the sensor-to-cloud pipeline delivered updates at the configured 5-second interval without triggering free-tier rate-limit warnings during testing.

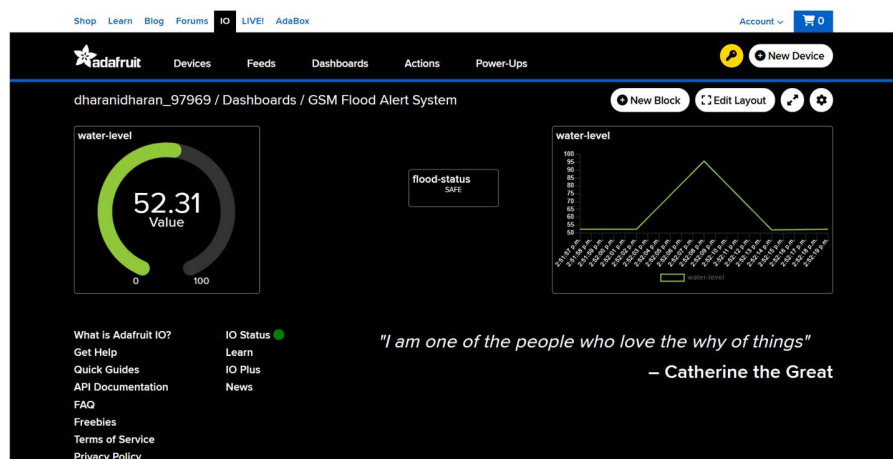


Figure 4: Adafruit IO dashboard during SAFE-condition testing — water-level value 52.31 and flood-status “SAFE”

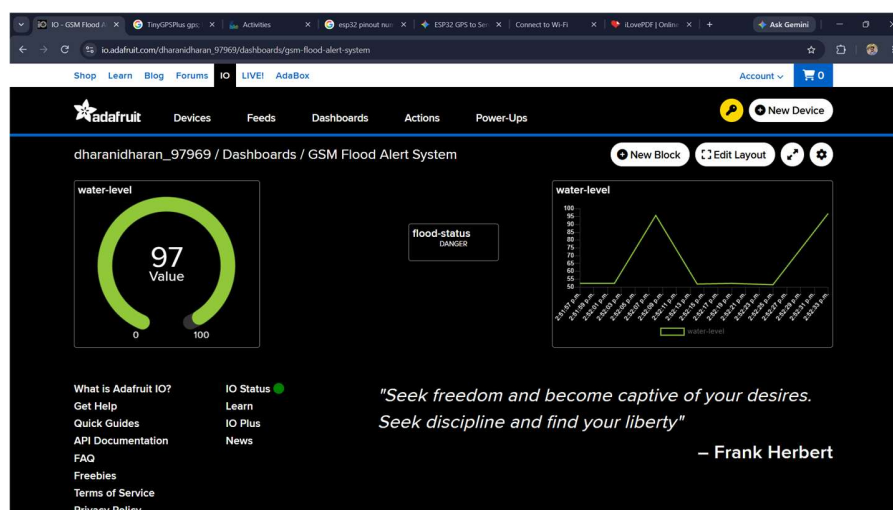


Figure 5: Adafruit IO dashboard during DANGER-condition testing — water-level value 97 and flood-status “DANGER”

Performance Summary

Parameter	Observed Result
Distance measurement	Consistent readings using the HC-SR04's 30 ms echo timeout, with sensor-error handling on missed echoes
SAFE / DANGER classification	Correctly switched at the 60 cm and 85 cm thresholds across repeated test runs
SMS alert behaviour	Sent exactly once per DANGER event; correctly suppressed on repeat DANGER readings via the alert latch
Data publish interval	5 seconds per update cycle (configurable via <code>CLOUD_UPDATE_INTERVAL</code>)
Cloud update latency	Dashboard gauge, status widget, and trend chart updated within a few seconds of each publish
Local feedback	LCD, LED, and buzzer responded instantaneously, independent of Wi-Fi/cloud latency

Conclusion

The prototype successfully demonstrates a working, low-cost water level detection and flood alert system. Water level was reliably measured using a non-contact ultrasonic sensor, classified into SAFE / WARNING / DANGER zones, and communicated through local LED/buzzer/LCD feedback, an SMS alert to a responsible person, and a remote Adafruit IO dashboard — giving early, actionable warning of rising water levels in rivers, tanks, or dams. Future improvements could include multi-point level averaging for noise rejection, solar-powered standalone operation for remote sites, multiple SMS recipients, and historical data logging for long-term flood-trend analysis.